

AI-Assisted Software Development

Understanding and Working with Language Models for Coding

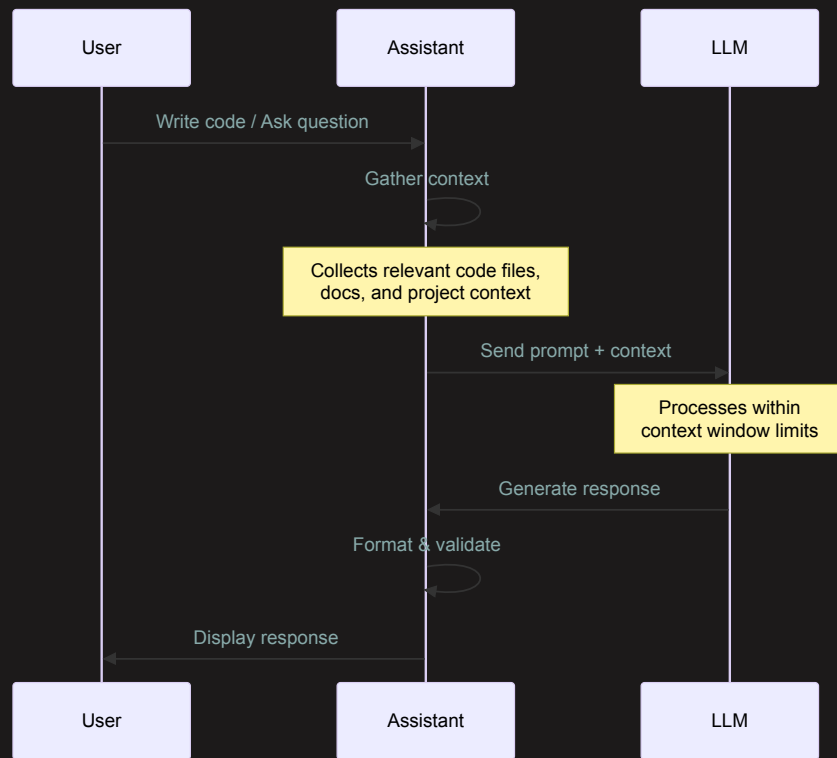
Mike Burton 4-6 November 2025

What are Code Assistants?

LLM-powered code assistants can:

- Generate code from descriptions
- Complete partial code
- Translate between languages
- Explain complex code
- Debug and fix issues
- Suggest refactoring
- Answer coding questions

How Code Assistants Work



Key Components:

1. Context Management

- Conversation history
- Code state tracking
- File dependencies

2. Prompt Engineering

- Code formatting
- Documentation inclusion
- Query structuring

3. Response Processing

- Code validation
- Output formatting
- Error handling

Understanding Context Windows

The context window is:

- Maximum text an LLM can process at once
- Includes both input (prompt) and output
- Measured in tokens (~3/4 of a word)
- Acts like the LLM's "working memory"

Common Window Sizes (2024)

- GPT-3.5: ~4K tokens
- Claude 2: ~100K tokens
- GPT-4: ~32K tokens
- Claude 3: ~200K tokens
- Gemini 1.5 Pro: ~1M tokens

Impact of Context Windows

Advantages of Larger Windows

- Can process entire codebases
- Better project context understanding
- More accurate refactoring
- Handles complex documentation
- Maintains multi-file context

Limitations

- Increased processing time
- Higher costs
- Greater memory usage
- May still miss context
- Requires efficient management

LLM Cut-off Date

What is it?

- Training data collection end date
- No knowledge of newer developments
- Varies by model (typically months to years old)

Impact on Code

1. Framework & Library Knowledge

- Missing newer frameworks
- Outdated best practices
- Deprecated approaches

2. API Understanding

- Outdated endpoints
- Missing parameters
- Deprecated features

Hallucinations in Code Assistance

What are They?

- Plausible but incorrect content
- Non-existent functions/APIs
- Fictional documentation
- Mixed framework features

Common Types

1. API Hallucinations

- Invented methods
- Fictional parameters
- Version mixing

2. Library Hallucinations

- Non-existent packages
- Incompatible features
- Made-up imports

Detecting Hallucinations

Red Flags

- Too-convenient solutions
- Missing documentation
- Inconsistent naming
- Perfect solutions
- Mixing framework features
- Vague completions eg "Look for"

Prevention

1. Always verify:

- Official documentation
- Package existence
- API endpoints
- Framework compatibility

2. Test thoroughly:

- Break into verifiable chunks
- Use automated testing
- Validate dependencies

3. Prompting strategies:

- Query / challenge to AI's results
- Suggest AI research online first, note differences
- Add "Dont guess"

Evaluating LLM Performance

Key Benchmarks

- HumanEval: Code generation from docstrings
- MBPP: Complex multi-file problems
- APPS: Problem-solving and debugging

Important Metrics

- Code correctness
- Completion speed
- Context understanding
- Documentation quality
- Edge case handling

Choosing a Language Model

Key Factors:

- Coding accuracy
- Response latency
- Cost per token
- Daily limits
- Context window
- Cut-off date
- API availability
- Data privacy (and IPR?)

Check leaderboards at:

- openrouter.ai/rankings
- aider.chat/docs/leaderboards

Cost Considerations

- Input token cost
- Output token cost
- Daily/monthly limits
- API access fees

Performance Trade-offs

- Accuracy vs Speed
- Cost vs Capability
- Window Size vs Price
- Availability vs Features

Best Practices

1. Context Management

- Include relevant files only
- Optimize context window usage
- Maintain conversation history

2. Verification

- Check official documentation
- Test generated code
- Verify dependencies
- Cross-reference sources

3. Development Workflow

- Use as starting point
- Break down complex tasks
- Implement review process
- Document verification steps

Turn now to Exercise, then Retrospective